CSS Appearance Property

Styling Form Controls & Dropdowns

Complete Guide: Removing & Customizing Defaults

Easy English • Practical Examples • Copy-Ready Code

Master CSS appearance property to customize select elements, checkboxes, radios, and buttons

Contents

Chapter 1

Introduction

1.1 What is the CSS Appearance Property?

The `appearance` property is a CSS feature that controls how form elements and buttons look by default. It lets you remove or replace the browser's default styling with your own custom design.

Definition

The `appearance` property removes native browser styling from form elements (like select dropdowns, checkboxes, and radio buttons) so you can style them completely from scratch using CSS.

1.2 Why Use the Appearance Property?

- **Consistency**: Make form elements look the same across all browsers
- **Branding**: Match your website's design language and colors
- **Customization**: Create unique, stylized form controls
- **Accessibility**: Build accessible custom controls
- **Modern Design**: Create forms that fit contemporary UI trends

1.3 The Problem with Default Styling

Different browsers render form elements differently. For example:

- Chrome's select dropdown looks different from Firefox's
- Checkboxes appear different on macOS vs. Windows
- Radio buttons have different styles across browsers

Solution: Use `appearance: none` to strip all default styling, then rebuild with CSS.

Chapter 2

Understanding the Appearance Property

2.1 What the Appearance Property Does

The `appearance` property tells the browser: "Stop using your default styling for this element." This gives you full control over the design.

2.2 Common Appearance Values

Value	Effect
`none`	Remove all default styling
`auto`	Use browser defaults (default)
`button`	Style like a native button
`checkbox`	Style like a native checkbox
`radio`	Style like a native radio button
`menulist`	Style like a dropdown menu

2.3 Browser Support

Browser	Support	Prefix
Chrome/Edge	Full	`-webkit-` or none
Firefox	Full	`-moz-` or none
Safari	Full	`-webkit-`
Opera	Full	`-webkit-`
Mobile Browsers	Full	Both

2.4 Vendor Prefixes

For maximum compatibility, use both prefixed and standard versions:

```
select {
    -webkit-appearance: none;  /* Safari, Chrome, Edge */
    -moz-appearance: none;     /* Firefox */
    appearance: none;          /* Standard (all browsers) */
}
```

Chapter 3

Styling Select Dropdowns

3.1 The Challenge

Select dropdowns are notoriously difficult to style. Browsers protect them for consistency and accessibility. Solution: Remove default styling and rebuild.

3.2 Basic Select Styling

3.2.1 Step 1: Remove Default Appearance

```
select {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;
}
```

3.2.2 Step 2: Add Custom Styling

```
select {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    /* Styling */
    width: 200px;
    padding: 10px;
    background-color: white;
    border: 2px solid #667eea;
    border-radius: 6px;
    color: #333;
    font-size: 14px;
    cursor: pointer;
}

select:hover {
    border-color: #764ba2;
}
```

```
select:focus {
    outline: none;
    border-color: #764ba2;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}
```

3.2.3 Step 3: Add the Dropdown Arrow

Since we removed the default arrow, add a custom one using CSS:

```
select {
    background-image: url('data:image/svg+xml;charset=UTF-8,%3csvg...');
    background-repeat: no-repeat;
    background-position: right 10px center;
    background-size: 20px;
    padding-right: 35px;  /* Space for the arrow */
}
```

3.3 Complete Custom Select Example

```
.custom-select {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 100%;
    padding: 12px 40px 12px 15px;
    border: 2px solid #ddd;
    border-radius: 8px;
    background-color: white;
    color: #333;
    font-size: 14px;
    font-weight: 500;
    cursor: pointer;

    /* Add custom arrow */
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
        /2000/svg' width='12' height='12' viewBox='0 0 12 12'%3E%3Cpath fill
        ='%23667eea' d='M6 9L1 4h10z'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 12px center;

    transition: all 0.3s ease;
}

.custom-select:hover {
    border-color: #667eea;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
        /2000/svg' width='12' height='12' viewBox='0 0 12 12'%3E%3Cpath fill
        ='%23764ba2' d='M6 9L1 4h10z'/%3E%3C/svg%3E");
```

```
}

.custom-select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 4px rgba(102, 126, 234, 0.15);
}

.custom-select:disabled {
    background-color: #f5f5f5;
    color: #999;
    cursor: not-allowed;
}
```

3.4 HTML Structure

```
<select class="custom-select">
    <option value="">Select an option</option>
    <option value="1">Option 1</option>
    <option value="2">Option 2</option>
    <option value="3">Option 3</option>
</select>
```

Chapter 4

Styling Checkboxes

4.1 Default Checkbox Problem

Default checkboxes are tiny, platform-specific, and hard to style. Most modern websites use custom designs.

4.2 Creating Custom Checkboxes

4.2.1 Basic Setup

```
input[type="checkbox"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 20px;
    height: 20px;
    border: 2px solid #ddd;
    border-radius: 4px;
    background-color: white;
    cursor: pointer;
    position: relative;
    transition: all 0.3s ease;
}

input[type="checkbox"]:hover {
    border-color: #667eea;
}

input[type="checkbox"]:checked {
    background-color: #667eea;
    border-color: #667eea;
}
```

4.2.2 Adding the Checkmark

```
input[type="checkbox"]:checked::after {
```

```
    content: '';
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    color: white;
    font-size: 14px;
    font-weight: bold;
}
```

4.3 Complete Checkbox Example

```
input[type="checkbox"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 22px;
    height: 22px;
    border: 2px solid #ccc;
    border-radius: 4px;
    background-color: white;
    cursor: pointer;
    position: relative;
    vertical-align: middle;
    transition: all 0.2s ease;
}

input[type="checkbox"]:hover {
    border-color: #667eea;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

input[type="checkbox"]:focus {
    outline: none;
    border-color: #667eea;
}

input[type="checkbox"]:checked {
    background-color: #667eea;
    border-color: #667eea;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' viewBox='0 0 20 20'%3E%3Cpath fill='white' d='M7 10l2 2
       4-4'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: center;
}

input[type="checkbox"]:disabled {
    background-color: #f5f5f5;
    border-color: #ddd;
    cursor: not-allowed;
```

```
}
```

4.4 HTML Usage

```
<label>
    <input type="checkbox" name="agree">
    I agree to the terms
</label>
```

4.5 Making Checkboxes Bigger

For better touch targets on mobile:

```
input[type="checkbox"] {
    width: 24px;
    height: 24px;
    /* Ensure 44x44px touch target with padding */
}

label {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 10px;
    cursor: pointer;
}
```

Chapter 5

Styling Radio Buttons

5.1 Radio Button Challenges

Like checkboxes, radio buttons are hard to style and look different across browsers.

5.2 Custom Radio Buttons

5.2.1 Basic Styling

```
input[type="radio"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 20px;
    height: 20px;
    border: 2px solid #ddd;
    border-radius: 50%;  /* Circle instead of square */
    background-color: white;
    cursor: pointer;
    transition: all 0.3s ease;
}

input[type="radio"]:hover {
    border-color: #667eea;
}

input[type="radio"]:checked {
    border-color: #667eea;
    background: radial-gradient(circle, #667eea 0%, #667eea 40%, white 50%);
}
```

5.2.2 Alternative: Filled Circle

```
input[type="radio"]:checked {
    border-color: #667eea;
    background-color: #667eea;
```

```
    box-shadow: inset 0 0 0 4px white;
}
```

5.3 Complete Radio Button Example

```
input[type="radio"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 20px;
    height: 20px;
    border: 2px solid #ccc;
    border-radius: 50%;
    background-color: white;
    cursor: pointer;
    vertical-align: middle;
    margin-right: 8px;
    transition: all 0.2s ease;
}

input[type="radio"]:hover {
    border-color: #667eea;
}

input[type="radio"]:focus {
    outline: none;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.2);
}

input[type="radio"]:checked {
    border-color: #667eea;
    background:
        radial-gradient(circle at center,
            #667eea 0%,
            #667eea 30%,
            white 31%);
}

input[type="radio"]:disabled {
    border-color: #ddd;
    background-color: #f5f5f5;
    cursor: not-allowed;
}

label {
    display: flex;
    align-items: center;
    cursor: pointer;
}
```

5.4 HTML Usage

```
<fieldset>
    <legend>Choose an option:</legend>

    <label>
        <input type="radio" name="choice" value="a">
        Option A
    </label>

    <label>
        <input type="radio" name="choice" value="b">
        Option B
    </label>
</fieldset>
```

Chapter 6

The ::picker(select) Pseudo-Element

6.1 What is ::picker(select)?

The `::picker(select)` is a CSS pseudo-element (still experimental) that specifically targets the dropdown picker part of `<select>` elements. It allows fine-grained control over just the button that opens the dropdown.

Warning

`::picker(select)` is still experimental and has limited browser support. Use with caution and provide fallbacks.

6.2 Current Browser Support

Browser	Status
Chrome	Experimental
Edge	Experimental
Firefox	Experimental
Safari	Not Supported

6.3 Basic Syntax

```
/* Target the picker button specifically */
select::picker {
    color: #667eea;
    background-color: #f5f5f5;
}

/* Alternative syntax for some browsers */
select::picker(select) {
    /* styles */
}
```

6.4 Practical Example

```
select {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 200px;
    padding: 10px;
}

/* Style the picker button (experimental) */
select::picker {
    color: #667eea;
    font-weight: bold;
}

select:hover::picker {
    color: #764ba2;
}
```

6.5 Limitations

- **Browser Support**: Very limited, mostly experimental
- **Styling Scope**: Can only style the button, not the dropdown options
- **Unpredictability**: Behavior varies across browsers
- **Fallback Needed**: Must use `appearance: none` as fallback

6.6 Recommendation

For now, **don't rely on `::picker(select)`**. Instead:

1. Use `appearance: none` to remove defaults
2. Style the entire select element with background-image for arrows
3. Use JavaScript libraries if you need full dropdown customization
4. Keep checking for updates as browser support improves

Chapter 7

Real-World Examples

7.1 Example 1: Modern Registration Form

```
.form-group select,
.form-group input[type="checkbox"],
.form-group input[type="radio"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;
}

select {
    width: 100%;
    padding: 12px;
    border: 1px solid #ddd;
    border-radius: 6px;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' width='12' height='8' viewBox='0 0 12 8'%3E%3Cpath fill
       ='%23667eea' d='M1 1l5 5 5-5'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 12px center;
    padding-right: 35px;
}

input[type="checkbox"] {
    width: 18px;
    height: 18px;
    cursor: pointer;
}

input[type="checkbox"]:checked {
    background-color: #667eea;
    border-color: #667eea;
}
```

7.2 Example 2: E-Commerce Filter Menu

```
.filter-select {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 180px;
    padding: 10px 12px;
    border: 2px solid #f0f0f0;
    border-radius: 4px;
    background-color: white;
    font-size: 13px;

    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' width='10' height='6' viewBox='0 0 10 6'%3E%3Cpath fill
       ='%23999' d='M1 1l4 4 4-4'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 8px center;
    padding-right: 28px;
}

.filter-select:hover {
    border-color: #ddd;
}

.filter-select:focus {
    outline: none;
    border-color: #667eea;
}
```

7.3 Example 3: Settings Page with Checkboxes

```
.settings-checkbox {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 40px;
    height: 24px;
    border-radius: 12px;
    border: 2px solid #ddd;
    background-color: white;
    cursor: pointer;
    position: relative;
    transition: all 0.3s ease;
}

.settings-checkbox:checked {
    background-color: #28a745;
    border-color: #28a745;
}
```

```
.settings-checkbox::after {
    content: '';
    position: absolute;
    top: 2px;
    left: 2px;
    width: 16px;
    height: 16px;
    border-radius: 50%;
    background-color: white;
    transition: left 0.3s ease;
}

.settings-checkbox:checked::after {
    left: 14px;
}
```

7.4 Example 4: Survey Form with Radio Buttons

```
.survey-radio {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 20px;
    height: 20px;
    border: 2px solid #ddd;
    border-radius: 50%;
    cursor: pointer;
    margin-right: 10px;
    transition: all 0.2s;
}

.survey-radio:hover {
    border-color: #667eea;
}

.survey-radio:checked {
    border-color: #667eea;
    background: radial-gradient(
        circle at center,
        #667eea 0%,
        #667eea 35%,
        white 36%
    );
}

.survey-label {
    display: flex;
    align-items: center;
    padding: 10px;
    margin-bottom: 10px;
    border-radius: 4px;
```

```
    cursor: pointer;
    transition: background 0.2s;
}

.survey-label:hover {
    background-color: #f9f9f9;
}
```

Chapter 8

Accessibility Considerations

8.1 Important: Maintain Usability

When customizing form controls, ensure they remain:

- **Visible**: Users must see which option is selected
- **Accessible**: Keyboard navigation must work
- **Understandable**: It must be obvious what's interactive

8.2 Focus States

Always include clear focus states:

```
select:focus,
input[type="checkbox"]:focus,
input[type="radio"]:focus {
    outline: none;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.3);
}
```

8.3 High Contrast Mode

Support users with vision impairments:

```
@media (prefers-contrast: more) {
    select,
    input[type="checkbox"],
    input[type="radio"] {
        border-width: 3px;
    }
}
```

8.4 Disabled State

Always style disabled controls differently:

```
input:disabled {
    background-color: #f5f5f5;
    color: #999;
    cursor: not-allowed;
    opacity: 0.6;
}
```

8.5 Keyboard Navigation

Test with keyboard alone:

- `Tab` to navigate between controls
- `Enter/Space` to toggle checkboxes/radios
- `Arrow keys` to navigate select options

8.6 Labels

Always use proper labels:

```
<!-- Good: Associated label -->
<label for="agree">
    <input id="agree" type="checkbox" name="agree">
    I agree to the terms
</label>

<!-- Bad: No association -->
<input type="checkbox">
<span>I agree</span>
```

Chapter 9

Common Issues & Solutions

9.1 Issue 1: Select Arrow Not Showing

Problem: Custom arrow doesn't appear.

Solution: Ensure background-image is set and padding provides space:

```
select {
    background-image: url(...);
    background-repeat: no-repeat;
    background-position: right 10px center;
    padding-right: 35px;  /* Space for arrow */
}
```

9.2 Issue 2: Appearance Property Not Working

Problem: Styles aren't applied to form control.

Solution: Use vendor prefixes and correct syntax:

```
input[type="checkbox"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;
}
```

9.3 Issue 3: Mobile Keyboard Doesn't Appear

Problem: Custom select doesn't trigger browser picker on mobile.

Solution: This is expected behavior. Mobile browsers use native controls. It's actually better for UX!

9.4 Issue 4: Focus Ring Doesn't Show

Problem: Users can't see when a form control has focus.

Solution: Always add visible focus states:

```
input:focus {
    outline: 2px solid #667eea;
    outline-offset: 2px;
}
```

9.5 Issue 5: Selected Text Hard to Read

Problem: Option text is difficult to read in custom select.

Solution: Improve contrast and readability:

```
select {
    color: #333;
    background-color: white;
    font-size: 14px;
}

select option {
    color: black;
    background-color: white;
}
```

Chapter 10

Advanced Techniques

10.1 Animated Dropdown Arrow

Rotate arrow when select is focused:

```
select {
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' width='12' height='8' viewBox='0 0 12 8'%3E%3Cpath fill
       ='%23667eea' d='M1 1l5 5 5-5'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 10px center;
    transition: background-image 0.3s ease;
}

select:focus {
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' width='12' height='8' viewBox='0 0 12 8' style='transform:
       rotate(180deg)'%3E%3Cpath fill='%23667eea' d='M1 1l5 5 5-5'/%3E%3C/svg
       %3E");
}
```

10.2 Gradient Checkbox

Advanced checkbox with gradient:

```
input[type="checkbox"]:checked {
    background: linear-gradient(135deg, #667eea, #764ba2);
    border-color: transparent;
}
```

10.3 Icon-Based Radio Button

Use custom icons for radio buttons:

```
input[type="radio"] {
    position: absolute;
    opacity: 0;
}
```

```
input[type="radio"] + label::before {
    content: '';
    display: inline-block;
    width: 24px;
    height: 24px;
    border: 2px solid #ddd;
    border-radius: 50%;
    margin-right: 10px;
    vertical-align: middle;
}

input[type="radio"]:checked + label::before {
    background-color: #667eea;
    border-color: #667eea;
}
```

10.4 Select with Icon

Add an icon inside the select element:

```
.select-with-icon {
    position: relative;
    display: inline-block;
}

.select-with-icon::before {
    content: '';
    position: absolute;
    right: 8px;
    top: 50%;
    transform: translateY(-50%);
    pointer-events: none;
}

.select-with-icon select {
    padding-right: 30px;
}
```

Chapter 11

Copy-Ready Code Templates

11.1 Template 1: Basic Custom Select

```
select {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 100%;
    padding: 10px 35px 10px 12px;
    border: 1px solid #ddd;
    border-radius: 4px;
    background-color: white;
    color: #333;
    font-size: 14px;
    cursor: pointer;

    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' width='12' height='8' viewBox='0 0 12 8'%3E%3Cpath fill
       ='%23999' d='M1 1l5 5 5-5'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 10px center;
    background-size: 12px;
}

select:hover {
    border-color: #666;
}

select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}
```

11.2 Template 2: Modern Checkbox

```
input[type="checkbox"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 20px;
    height: 20px;
    border: 2px solid #ddd;
    border-radius: 4px;
    background-color: white;
    cursor: pointer;
    transition: all 0.2s ease;
}

input[type="checkbox"]:hover {
    border-color: #667eea;
}

input[type="checkbox"]:checked {
    background-color: #667eea;
    border-color: #667eea;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' viewBox='0 0 16 16'%3E%3Cpath fill='white' d='M3 8l2 2
       4-4'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: center;
}

input[type="checkbox"]:focus {
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.2);
}
```

11.3 Template 3: Radio Button Group

```
input[type="radio"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;

    width: 20px;
    height: 20px;
    border: 2px solid #ddd;
    border-radius: 50%;
    background-color: white;
    cursor: pointer;
    vertical-align: middle;
    margin-right: 8px;
    transition: all 0.2s ease;
}

input[type="radio"]:hover {
```

```
    border-color: #667eea;
}

input[type="radio"]:checked {
    border-color: #667eea;
    background: radial-gradient(
        circle at center,
        #667eea 0%,
        #667eea 35%,
        white 36%
    );
}

input[type="radio"]:focus {
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.2);
}

label {
    display: flex;
    align-items: center;
    cursor: pointer;
    margin-bottom: 10px;
}
```

11.4 Template 4: Premium Form Set

```
/* All form controls */
select,
input[type="checkbox"],
input[type="radio"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    appearance: none;
}

/* Select dropdown */
select {
    width: 100%;
    padding: 12px 40px 12px 15px;
    border: 2px solid #e0e0e0;
    border-radius: 6px;
    background-color: #fff;
    color: #333;
    font-size: 14px;
    cursor: pointer;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org
       /2000/svg' width='12' height='8' viewBox='0 0 12 8'%3E%3Cpath fill
       ='%23667eea' d='M1 1l5 5 5-5'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 12px center;
    transition: all 0.3s ease;
}
```

```
select:hover { border-color: #667eea; }
select:focus {
    outline: none;
    border-color: #667eea;
    box-shadow: 0 0 0 4px rgba(102, 126, 234, 0.1);
}

/* Checkbox */
input[type="checkbox"] {
    width: 20px;
    height: 20px;
    border: 2px solid #e0e0e0;
    border-radius: 4px;
    background-color: white;
    cursor: pointer;
    transition: all 0.2s ease;
}

input[type="checkbox"]:checked {
    background-color: #667eea;
    border-color: #667eea;
}

/* Radio button */
input[type="radio"] {
    width: 20px;
    height: 20px;
    border: 2px solid #e0e0e0;
    border-radius: 50%;
    background-color: white;
    cursor: pointer;
    transition: all 0.2s ease;
}

input[type="radio"]:checked {
    border-color: #667eea;
    background: radial-gradient(circle at center, #667eea 0%, #667eea 30%,
       white 31%);
}
```

Chapter 12

Browser Compatibility Details

12.1 Appearance Property Support

Browser	Support Level	Notes
Chrome 60+	Full	Uses `-webkit-appearance`
Firefox 54+	Full	Uses `-moz-appearance`
Safari 5.1+	Full	Uses `-webkit-appearance`
Edge 79+	Full	Uses `-webkit-appearance`
IE 11	None	Not supported

12.2 Recommended Prefix Strategy

```
/* For maximum compatibility */
input[type="checkbox"] {
    -webkit-appearance: none;
    -moz-appearance: none;
    -ms-appearance: none;
    appearance: none;
}
```

12.3 Feature Detection

Check if appearance is supported:

```
function supportsAppearance() {
    const element = document.createElement('input');
    element.type = 'checkbox';
    element.style.webkitAppearance = 'none';
    return element.style.webkitAppearance !== '';
}

if (supportsAppearance()) {
    document.body.classList.add('appearance-supported');
}
```

Chapter 13

Best Practices & Guidelines

13.1 Do This

- Always use `appearance: none` with all vendor prefixes
- Include clear, visible focus states
- Make interactive elements at least 44x44px (mobile touch target)
- Use semantic HTML (`<label>` with inputs)
- Test with keyboard navigation
- Provide visual feedback on hover and active states
- Maintain good color contrast ratios
- Include disabled state styling

13.2 Don't Do This

- Don't hide form controls completely with `display: none`
- Don't make controls too small (less than 40px)
- Don't remove focus indicators without replacement
- Don't use appearance without fallbacks
- Don't assume selected/unselected states are obvious
- Don't ignore keyboard navigation
- Don't use low-contrast colors
- Don't forget about disabled states

13.3 Testing Checklist

- Test in Chrome, Firefox, Safari, and Edge
- Keyboard navigation with Tab key
- Enter/Space to select/toggle
- Mouse hover states
- Focus visible with keyboard
- Disabled state styling
- Mobile responsiveness
- Screen reader compatibility

Chapter 14

Summary & Key Takeaways

14.1 What You've Learned

- The CSS `appearance` property removes browser defaults
- Use `appearance: none` with vendor prefixes
- Customize select dropdowns with background-image arrows
- Create modern checkboxes and radio buttons
- The experimental `::picker(select)` pseudo-element
- Accessibility requirements for custom form controls
- Browser compatibility and testing strategies
- Real-world examples and copy-ready code

14.2 Quick Reference Table

Element	Appearance Value
Select dropdown	`none`
Checkbox	`none`
Radio button	`none`
Button	`none`
Standard	`auto`

14.3 Next Steps

1. Start with simple `appearance: none` on one control
2. Add custom styling with CSS properties
3. Test across browsers and devices
4. Gather user feedback
5. Iterate and refine
6. Consider accessibility from the start

14.4 When to Use Custom Controls

Use custom styling when:

- Your design requires specific branding
- Default controls don't match the aesthetic
- You need better UX/consistency
- You have resources for testing

Use defaults when:

- Defaults work fine for your use case
- You need minimal development time
- Mobile compatibility is critical
- Accessibility is high priority

Master Form Styling!

Custom form controls dramatically improve user experience when done right.